

PROJETO

Aplicativo Comedoria da Tia

INTEGRANTES DO PROJETO e RA'S

Bianca Marcondes Camargo	-	25028292
Erik dos Santos Raimundo	-	24026628
Larissa de Almeida Lira Oliveira	-	24026849
Vinicius Cardoso de Lima	-	24026851

São Paulo

2025

Sumário

1. INTRODUÇÃO.....	4
2. Qualidade de Software.....	4
2.1. Modelo que qualidade de software (Diagrama/Design)	4
2.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo)	4
2.3. Identificação de atributos de qualidade da norma 25010.....	4
2.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.	4
3. Teste de Software	4
3.1. Plano de Teste	4
3.2. Apresentar 2 testes unitários.	4
3.3. Apresentar 2 testes de integração	5
3.4. Apresentar um teste de usuário (SISTEMA)	5
4. REFERÊNCIAS BIBLIOGRÁFICAS	5

Data das entregas	22/set.	10/nov.	17/11 - 25/11
Disciplinas	Entrega 1 (3,0 Pontos)	Entrega 2 (4,0 Pontos)	Apresentação (3,0 Pontos)
Testes de Software	Instruções de Entrega: Qualidade de Software Descrição: O aluno deverá demonstrar o entendimento dos conceitos de qualidade de software, aplicando-os ao projeto. Isso inclui a criação de um diagrama do processo de qualidade de software, a identificação de atributos de qualidade. Apresentar um relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento. Entrega em documento pdv seguindo o template fornecido pelo professor.	Instruções de Entrega: Testes Unitários, de Integração e de usuário. Descrição: Os alunos deverão desenvolver 4 testes unitários e 2 testes de integração e 1 teste de usuário. Esses testes devem garantir que o sistema esteja funcionando corretamente e validando os principais componentes e integrações e a geração de um relatório de teste de carga do servidor. Entrega em documento pdv seguindo o template fornecido pelo professor.	Critérios: Criatividade, Impacto Social, Tempo de apresentação e Embasamento.

1. INTRODUÇÃO

Fome e pouco tempo? A tia te entende!

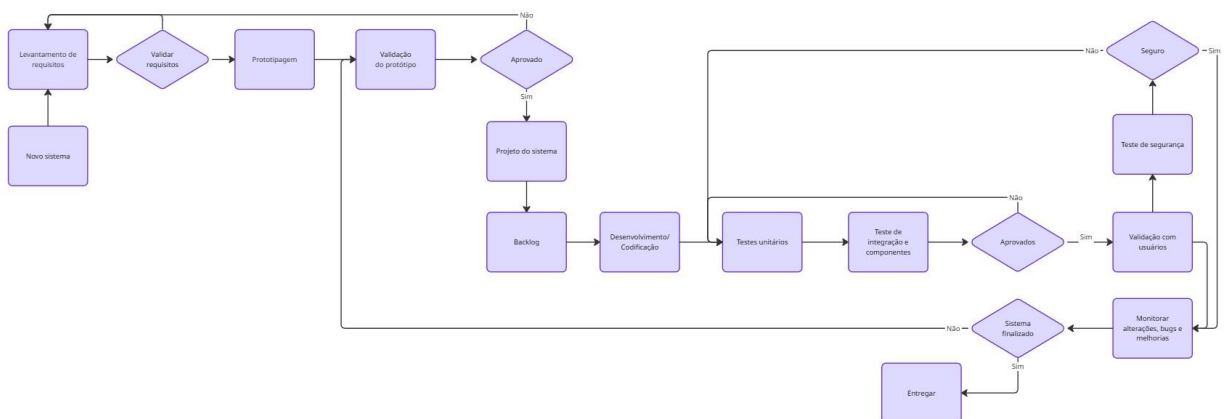
É por isso que o **Comedoria da Tia** foi criado! o seu novo aliado na hora de fazer uma refeição na faculdade. O aplicativo foi pensado para resolver os problemas que vocês já conhecem bem: as filas enormes no horário de pico e a correria de ir e voltar até o refeitório.

Com ele, vocês fazem seu pedido de forma prática e rápida, direto do seu celular, seja da sua sala de aula, da biblioteca ou de onde estiver. É só escolher o que quer comer, pagar pelo app e pronto! A gente avisa quando estiver tudo preparado para você só passar e retirar, **sem precisar enfrentar a fila**.

Assim, você aproveita ao máximo seu intervalo, garantindo uma refeição deliciosa sem estresse. Mais tempo para estudar, relaxar ou conversar com os amigos!

2. Qualidade de Software

2.1. Modelo de qualidade de software (Diagrama/Design)



2.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo)

Plano de Gerenciamento de Qualidade de Software

Com base no diagrama e na norma ISO/IEC 25010, que define as características de qualidade de produto de software, é possível estruturar um plano de gerenciamento de qualidade. O diagrama ilustra um ciclo de vida de desenvolvimento que incorpora etapas de validação e testes, fundamentais para garantir a qualidade do produto.

O processo começa com o Levantamento de Requisitos, seguido pela Validação de Requisitos e Prototipagem. A validação garante que as necessidades do usuário foram compreendidas corretamente, abordando a característica de adequação funcional da ISO 25010.

Se o protótipo for aprovado, o projeto avança para a fase de Projeto do Sistema e a criação do Backlog, que é a lista de funcionalidades a serem desenvolvidas.

A etapa de Desenvolvimento/Codificação é a implementação do projeto. Após a codificação, o processo se concentra em testes para garantir a qualidade.

1. **Testes Unitários:** Verificam a menor parte do código individualmente, focando na manutenibilidade e eficiência do software.
2. **Teste de Integração e Componentes:** Avaliam como diferentes módulos do software interagem, assegurando a adequação funcional e a compatibilidade.

Se os testes não forem aprovados, o processo retorna para a etapa de codificação e testes, formando um ciclo de correção e melhoria.

Após a aprovação, o processo foca na qualidade sob a perspectiva do usuário e da segurança.

1. **Teste de Segurança:** É uma etapa crucial para garantir que o software está seguro, protegendo dados e sistemas contra acesso não autorizado. Isso aborda diretamente a característica de segurança da ISO 25010.
2. **Validação com Usuários:** Nesta fase, o software é testado com os usuários finais. Isso garante que o produto atende aos requisitos de usabilidade e adequação funcional, confirmando que o software é eficiente, compreensível e satisfatório para o seu uso.

Finalmente, se o sistema for considerado finalizado, ele é entregue. No entanto, o diagrama sugere um processo contínuo de Monitoramento de alterações, bugs e melhorias, que é essencial para a manutenibilidade do software a longo prazo.

Em resumo, o diagrama demonstra um plano de qualidade que cobre as principais fases do desenvolvimento de software, com foco em:

- **Adequação Funcional:** Garantida pelo levantamento de requisitos, prototipagem e testes de integração.
- **Segurança:** Abordada pela etapa de testes de segurança dedicados.
- **Usabilidade:** Verificada diretamente com os usuários finais.
- **Manutenibilidade:** Assegurada pelos testes unitários e pelo monitoramento contínuo após a entrega.

Este plano alinha-se aos princípios da ISO 25010, que servem como guia para garantir que o software não apenas funcione, mas seja robusto, seguro e satisfaça as expectativas de seus usuários.

2.3. Identificação de atributos de qualidade da norma 25010.

Usabilidade

A usabilidade, que se refere à facilidade com que um usuário pode interagir com o sistema, é comprometida. O documento aponta que o acesso ao cardápio é difícil.

Eficiência de Desempenho

A eficiência de desempenho, que trata do comportamento do sistema em relação ao uso de recursos e ao tempo de resposta, é o principal problema. As filas grandes são um sintoma direto dessa ineficiência, causada principalmente pelo atraso em pagamentos.

Adequação Funcional

A adequação funcional, que se refere à capacidade do software de fornecer as funções necessárias para atender aos requisitos do usuário, apresenta deficiências. A falta de acesso ao estoque impede que o sistema informe sobre produtos em falta.

2.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.

A norma servirá como uma estrutura para garantir que o software atenda às necessidades do refeitório e resolva os problemas identificados, como **filas grandes** e a **distância** das salas de aula.

Adequação Funcional

A adequação funcional será o ponto de partida do desenvolvimento, garantindo que o app tenha as funcionalidades necessárias para resolver os problemas. O sistema atual falha em informar sobre **produtos em falta** por **não ter acesso a estoque**. O novo aplicativo resolverá isso ao fornecer uma função que exibe o estoque em tempo real. Além disso, para solucionar o problema da **distância** que leva à **escolha de outras cantinas**, o app oferecerá a capacidade de pré-encomenda e pagamento de qualquer lugar, permitindo que os alunos passem apenas para retirar seu pedido.

Usabilidade

A usabilidade se concentrará em criar uma experiência de usuário simples e intuitiva. Os problemas de **acesso difícil ao cardápio**, como o **QR code de difícil acesso**, serão eliminados. O design do aplicativo priorizará uma interface limpa e direta para a navegação do cardápio e para o processo de pagamento. A **separação de ComeDinheiro**, que complica o fluxo de pagamentos, será substituída por um método de pagamento integrado e unificado, facilitando a vida do usuário.

Eficiência de Desempenho

Para resolver as **filas grandes** e o **atraso em pagamentos**, a eficiência de desempenho será uma métrica-chave. O app será desenvolvido para otimizar as transações, garantindo que os **pagamentos em cartão** não sejam **lentos** e que o sistema possa lidar com a **alta demanda** em horários de pico. Isso permitirá que o refeitório processe pedidos de forma rápida, sem causar atrasos.

Confiabilidade

A confiabilidade do aplicativo será assegurada por meio de testes rigorosos. O problema de o **cardápio não carregar** será eliminado, garantindo que as funcionalidades essenciais do app, como a visualização do cardápio e o processamento de pedidos, operem de forma consistente e sem falhas em qualquer momento.

3. Teste de Software

3.1. Plano de Teste

1. Validação do Protótipo (Antes da Codificação)

Objetivo do Teste	Tipo de Teste	Cenário de Teste Simples	Resultado Esperado
Usabilidade do Protótipo (UX)	Teste de Usabilidade	Usuários (pais, alunos, cantineiro) navegam pelo <i>mockup</i> para fazer um pedido.	O fluxo de pedido é intuitivo, com poucos cliques e <i>feedback</i> claro.
Funcionalidade do Cardápio	Teste Funcional (em protótipo)	Verificar se as opções de filtro e busca no cardápio virtual são fáceis de usar e estão visíveis.	Opções de filtro por categoria (e.g., Lanches, Bebidas, Saudáveis) funcionam corretamente.

2. Testes Unitários e de Componentes (Durante a Codificação)

Objetivo do Teste	Etapa no Modelo	Cenário de Teste Simples	Foco Principal
-------------------	-----------------	--------------------------	----------------

Lógica de Cálculo de Pedidos	Testes Unitários	Verificar se a função de soma do carrinho calcula o total correto, incluindo impostos/taxas.	Componente de Cálculo de Preço.
Integração de Login	Teste de Integração e Componentes	Tentar fazer login com credenciais válidas e inválidas, verificando a comunicação com o Serviço de Autenticação.	Integração entre a interface de login e o backend de autenticação.
Transação de Pagamento	Teste de Integração e Componentes	Simular o envio de dados do pedido do carrinho para o Gateway de Pagamento.	Componente de Pagamento.

3. Validação com Usuários (Testes Funcionais e de Sistema)

Objetivo do Teste	Tipo de Teste	Cenário de Teste Simples	Dados de Teste
Realização de Pedido	Funcional (End-to-End)	Um aluno realiza um pedido completo, do login à confirmação, usando saldo pré-pago.	Usuário com saldo positivo.
Gestão de Estoque (Cantineiro)	Funcional	O cantineiro marca um item popular como "Indisponível" e verifica se ele some do cardápio do aluno.	Acesso do Cantineiro.
Visualização de Histórico	Funcional	O pai acessa o perfil e verifica o histórico de compras de seu filho.	Usuário "Pai" vinculado a um "Aluno".

4. Teste de Segurança (Validação de Segurança)

Objetivo do Teste	Tipo de Teste	Cenário de Teste Simples	Foco Principal
Injeção de Dados	Teste de Segurança	Inserir scripts maliciosos ou comandos em campos de texto (e.g., nome, observações do pedido).	Proteção contra injeção de código.

Transmissão de Dados	Teste de Segurança	Verificar se todos os dados sensíveis (senhas, dados de pagamento) são transmitidos via HTTPS e criptografados.	Criptografia e segurança da comunicação.
-----------------------------	--------------------	--	---

5. Monitoramento de Bugs e Melhorias (Após o Lançamento)

Objetivo do Teste	Tipo de Teste	Descrição	Ferramenta Típica
Regressão	Teste Pós-Correção	Após a correção de um <i>bug</i> (ex: falha no cálculo do troco), reexecutar todos os testes de pedidos para garantir que não surgiram novos problemas.	Ferramenta de Gerenciamento de Testes.
Monitoramento de Erros	Monitoramento	Acompanhar logs de erros e relatórios de falhas da aplicação em tempo real.	Ferramentas de <i>Application Performance Monitoring</i> (APM).

3.2. Apresentar 2 testes unitários.

Seguindo o plano de teste, realizamos o teste de lógica de cálculos de valores, onde utilizamos uma calculadora com valores declarados no Cardápio da Comedoria da Tia, adicionamos os valores aos produtos de forma dinâmica, usaremos o valor dos picolés como exemplo, com o código abaixo nós buscamos o valor da unidade de produto, e calculamos baseado em quantidade.

```
public class ItemCarrinho implements Serializable {
    private final Produto produto;
    private int quantidade;
    private final double precoUnitario;

    public ItemCarrinho(Produto produto, int quantidade) {
        this.produto = produto;
        this.quantidade = quantidade;
        this.precoUnitario = produto.getPrecoAtual();
    }

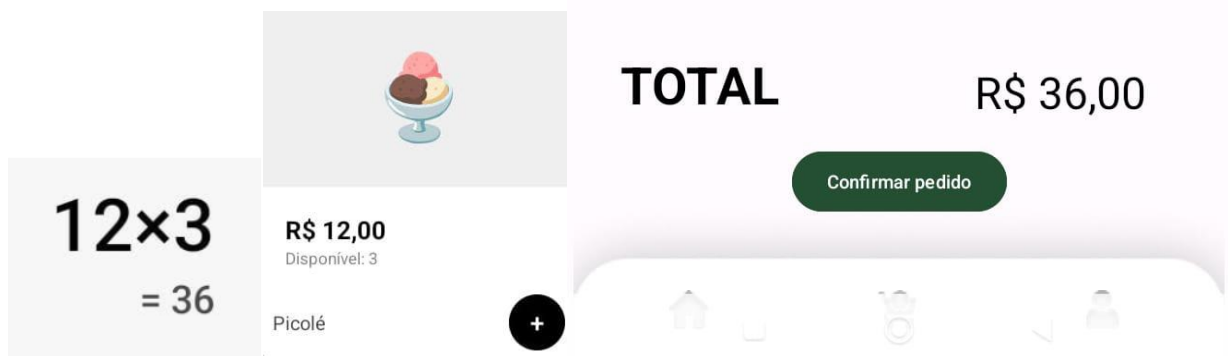
    public Produto getProduto() {
        return produto;
    }

    public int getQuantidade() {
        return quantidade;
    }

    public void setQuantidade(int quantidade) {
        this.quantidade = quantidade;
    }

    public double getPrecoUnitario() {
        return precoUnitario;
    }

    public double getPrecoTotal() {
        return precoUnitario * quantidade;
    }
}
```

A Seguir, realizamos o teste de integração de login, onde realizamos o login com credenciais válidas e inválidas e observamos como o sistema se comporta.

```
private void fazerLogin() {

    final String email = emailRaEditText.getText().toString().trim();
    final String senha = senhaEditText.getText().toString().trim();

    if (email.isEmpty() || senha.isEmpty()) {
        Toast.makeText(Login.this, "Preencha Email/RA e Senha.", Toast.LENGTH_SHORT).show();
        return;
    }

    String loginUrl = API_URL + "?email=eq." + email + "&password=eq." + senha;

    JSONArrayRequest request = new JSONArrayRequest(Request.Method.GET, loginUrl, null,

        new Response.Listener<JSONArray>() {

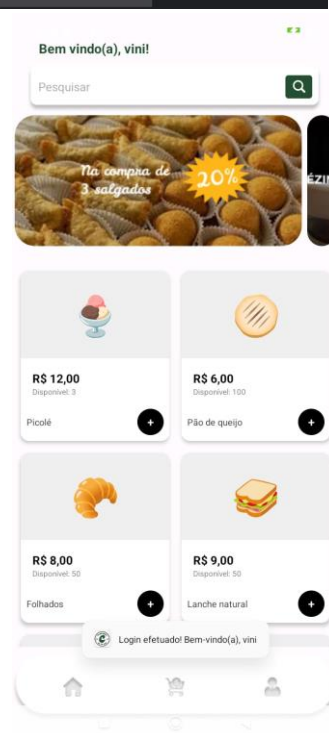
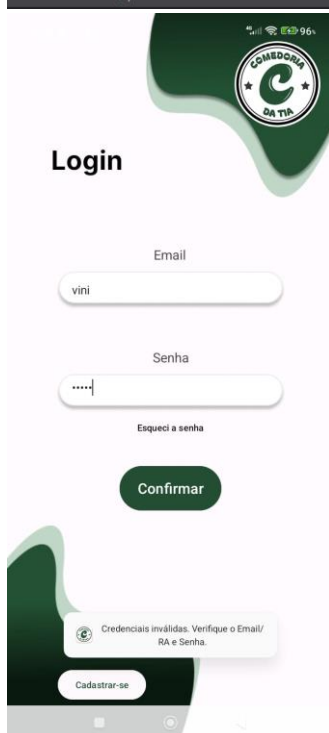
            @Override
            public void onResponse(JSONArray response) {

                if (response.length() > 0) {
                    try {
                        JSONObject user = response.getJSONObject(0);
                        String nomeUsuario = user.getString("name");

                        Toast.makeText(Login.this, "Login efetuado! Bem-vindo(a), " + nomeUsuario, Toast.LENGTH_LONG).show();

                        Intent irParaTelaInicial = new Intent(Login.this, TelaInicial.class);
                        irParaTelaInicial.putExtra("nomeUsuario", nomeUsuario);
                        startActivity(irParaTelaInicial);
                        finish();

                    } catch (JSONException e) {
                        e.printStackTrace();
                        Toast.makeText(Login.this, "Erro ao processar dados do usuário.", Toast.LENGTH_LONG).show();
                    }
                } else {
                    Toast.makeText(Login.this, "Credenciais inválidas. Verifique o Email/RA e Senha.", Toast.LENGTH_LONG).show();
                }
            }
        }
    );
}
```



Como apresentado, o sistema impede o acesso em caso de credenciais inválidas, e te permite o acesso ao app se suas credenciais sejam validadas pelo banco de dados.

3.3. Apresentar 2 testes de integração

Neste tópico iremos realizar dois testes de integração, sendo eles, o teste de gestão de estoque, e o teste de pagamento.

Teste de pagamento

```
no usages
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_pagamento);

    valorTotal = findViewById(R.id.valorTotal);
    rgPagamento = findViewById(R.id.rgPagamento);
    btnConfirmar = findViewById(R.id.btnConfirmar);
    cancelar = findViewById(R.id.cancelar);

    cancelar.setOnClickListener(v -> {
        finish();
    });

    String valor = getIntent().getStringExtra("total");
    if (valor != null) {
        double valorDouble = Double.parseDouble(valor);
        valorTotal.setText("Total " + String.format(Locale.getDefault(), "R$ %.2f", valorDouble));
    } else {
        valorTotal.setText("Total R$0.00");
    }

    rgPagamento.setOnCheckedChangeListener(new RadioGroup.OnCheckedChangeListener() {
        no usages
        @Override
        public void onCheckedChanged(RadioGroup group, int checkedId) {
            if (checkedId != -1) {
                RadioButton selectedRadioButton = findViewById(checkedId);
                metodoPagamentoSelecioneado = selectedRadioButton.getText().toString();
                Log.d("PAGAMENTO", "Método selecionado: " + metodoPagamentoSelecioneado);
            } else {
                metodoPagamentoSelecioneado = "Não selecionado";
            }
        }
    });

    btnConfirmar.setOnClickListener(new View.OnClickListener() {
        no usages
        @Override
        public void onClick(View v) {
            if (rgPagamento.getCheckedRadioButtonId() == -1) {
                Toast.makeText(Pagamento.this, "Por favor, selecione uma forma de pagamento.", Toast.LENGTH_SHORT).show();
            } else {
                String mensagem = "Pagamento de " + valorTotal.getText().toString() +
                    " confirmado via " + metodoPagamentoSelecioneado + ".";
                Toast.makeText(Pagamento.this, mensagem, Toast.LENGTH_LONG).show();
            }
        }
    });
}
```




Formas de pagamento

Cancelar compra

☐ Crédito
VISA, MASTERCARD, ELO, HIPERCARD, VR

☐ Débito
VISA, MASTERCARD, ELO, HIPERCARD, VR

☐ Pix
CHAVE PIX: 123.456.789-10

☐ Dinheiro
PAGAR NO BALCÃO

Total R\$ 12,00

Confirmar pagamento

Formas de pagamento

Cancelar compra

☐ Crédito
VISA, MASTERCARD, ELO, HIPERCARD, VR

☐ Débito
VISA, MASTERCARD, ELO, HIPERCARD, VR

☐ Pix
CHAVE PIX: 123.456.789-10

☐ Dinheiro
PAGAR NO BALCÃO

Por favor, selecione uma forma de pagamento.

Tc

Confirmar pagamento

Formas de pagamento

Cancelar compra

☐ Crédito
VISA, MASTERCARD, ELO, HIPERCARD, VR

☐ Débito
VISA, MASTERCARD, ELO, HIPERCARD, VR

☐ Pix
CHAVE PIX: 123.456.789-10

☒ Dinheiro
PAGAR NO BALCÃO

Pagamento de Total R\$ 12,00 confirmado via Dinheiro.

Tc

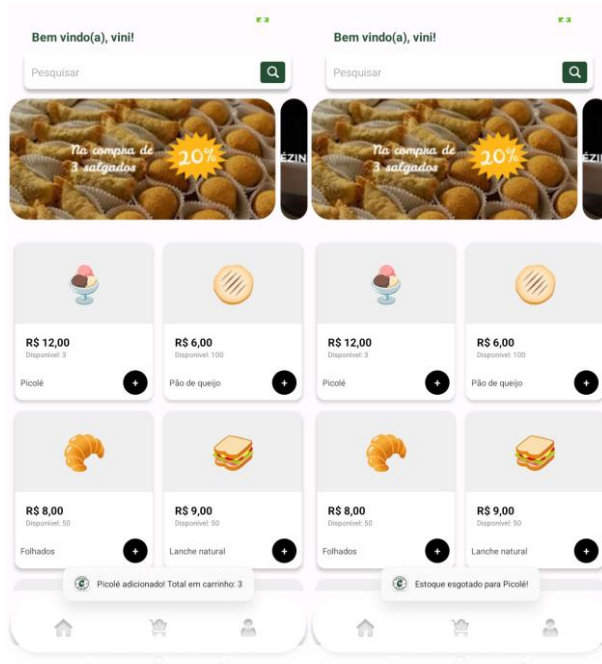
Confirmar pagamento

Como apresentado tanto em código, quanto em sistema, o software não permite seguir com pagamento sem que um método seja selecionado, assim que um método é selecionado, nosso protótipo libera para o banco de dados que o pedido foi realizado.

Teste de Gestão de estoque.

```
@Override
public void onAdicionarClick(Produto produto) {
    String nomeProduto = produto.getNome();
    int quantidadeEmCarrinho = carrinhoItens.getOrDefault(nomeProduto, 0);

    if (produto.getEstoque() > quantidadeEmCarrinho) {
        carrinhoItens.put(nomeProduto, quantidadeEmCarrinho + 1);
        Toast.makeText(this, nomeProduto + " adicionado! Total em carrinho: " + (quantidadeEmCarrinho + 1), Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "Estoque esgotado para " + nomeProduto + "!", Toast.LENGTH_SHORT).show();
    }
}
}
```



A quantidade em estoque é predefinida ao início das vendas, podendo ser alterado pelo web app disponível apenas aos funcionários da cantina, assim que um produto esgota, o software exibe um alerta de esgotamento, impedindo que o usuário adicione produtos esgotados ao carrinho.

3.4. Apresentar um teste de usuário (SISTEMA)

Para a apresentação do teste de usuário, entramos em um acordo de exibir um vídeo de testes do protótipo, nesse vídeo utilizamos todos os recursos apresentados acima, exibimos todas as telas, realizamos uma compra e exibimos a diferença do modo escuro para o claro.

Segue abaixo o vídeo e a descrição do vídeo:

<https://youtube.com/shorts/u0D2gOBtVf8?si=AK16sqW7esTMMMUu>

Descrição da Execução no Vídeo

Durante o vídeo, tanto no **tema escuro** quanto no **tema claro**, executamos os seguintes passos:

1. Acessamos a **página de cadastro**.
2. Acessamos um login já existente com as **credenciais inválidas**.
3. Acessamos um login com as **credenciais válidas**.
4. Após a verificação com o banco de dados, o **cardápio é acessado**.
5. Apresentamos a página de **carrinho enquanto vazia**.
6. Exibimos as funções de filtro por pesquisa, adição e remoção de produtos ao carrinho.
7. A **soma dos valores** é exibida antes e durante o processo de pagamento.
8. Por fim, exibimos a **tela de perfil**.

4. REFERÊNCIAS BIBLIOGRÁFICAS

<https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>

SOMMERVILLE, I. **Engenharia de Software**. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.